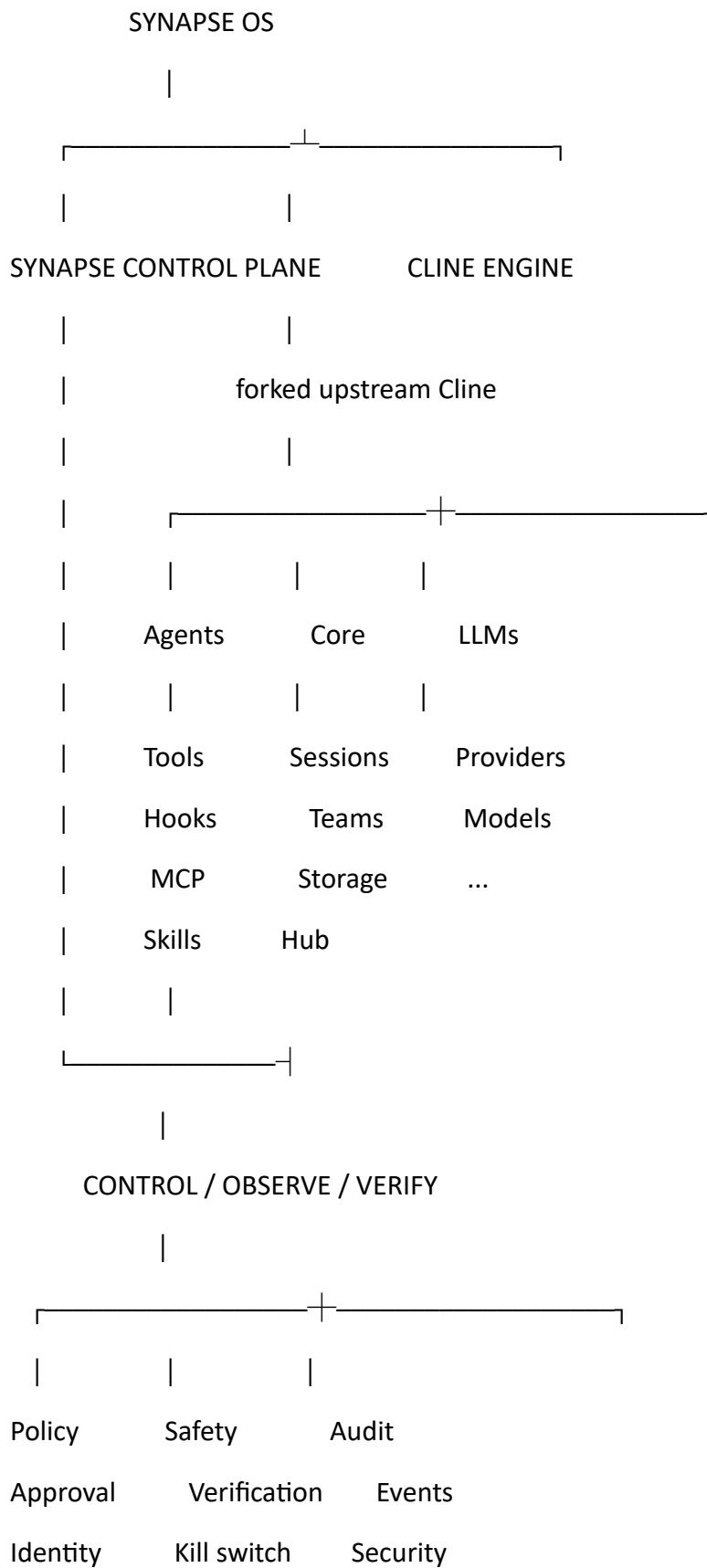
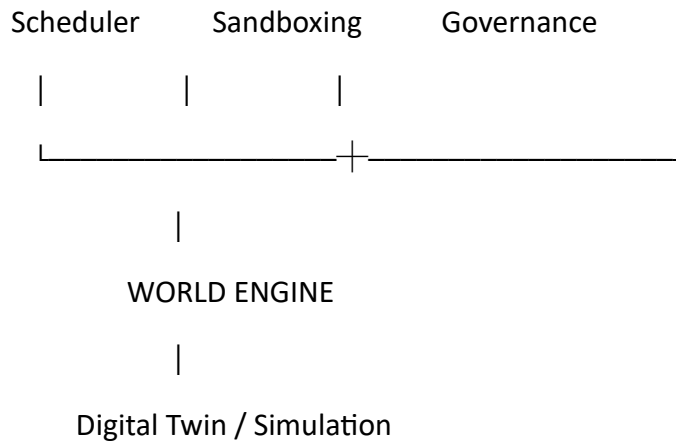


Synapse OS — Backend Implementation-Level File Manifest

0. The fundamental architecture





Important: Synapse should **not fork Cline** and then rewrite Cline's internals unnecessarily.

Instead:

upstream Cline



minimal Synapse patches



Cline Engine



Synapse Adapter



Synapse Control Plane

This makes upstream rebasing possible.

Cline's own architecture already puts stateful orchestration in `@cline/core`, the agent loop in `@cline/agents`, provider handling in `@cline/llms`, and shared contracts in `@cline/shared`.

1. Repository root

I would structure the fork like this:

synapse/

|

├─ apps/

| └─ backend/

- | └─ frontend/
- | └─ worker/
- |
- | └─ packages/
- | └─ control-plane/
- | └─ engine-adapter/
- | └─ policy-engine/
- | └─ approval-engine/
- | └─ verification-engine/
- | └─ safety-engine/
- | └─ audit-engine/
- | └─ event-bus/
- | └─ agent-registry/
- | └─ runtime-manager/
- | └─ scheduler/
- | └─ connector-manager/
- | └─ world-engine/
- | └─ simulation-engine/
- | └─ twin-engine/
- | └─ security/
- | └─ secrets/
- | └─ tenancy/
- | └─ observability/
- | └─ contracts/
- | └─ database/
- |
- | └─ engine/

```
| └─ cline/
|
|─ migrations/
|─ scripts/
|─ tests/
|─ docker/
|─ docs/
|
|─ package.json
|─ bun.lock
|─ turbo.json
|─ docker-compose.yml
└─ README.md
```

The important distinction:

engine/cline/

is the **upstream Cline source**.

Everything under:

packages/

apps/

is **Synapse code**.

2. engine/cline/

Do not flatten Cline into Synapse.

Preserve its structure as much as possible.

engine/cline/

```
|
|─ sdk/
```

```

|   ├── packages/
|   |   ├── shared/
|   |   ├── llms/
|   |   ├── agents/
|   |   ├── core/
|   |   ├── sdk/
|   |   └── ui/
|   |
|   └── examples/
|
└── apps/
    ├── cli/
    ├── cline-hub/
    ├── vscode/
    └── examples/
|
└── docs/
    ├── scripts/
    ├── package.json
    └── ...

```

Cline's current monorepo actually uses these package boundaries and workspace structure.

Rule

Do **not** put Synapse business logic into:

engine/cline/sdk/packages/core/

unless the functionality genuinely belongs to Cline itself.

Instead:

Cline → emits/returns capability

Synapse → governs capability

3. What Synapse owns vs Cline owns

This boundary is critical.

Capability	Owner
Agent reasoning loop	Cline
Tool execution	Cline
Filesystem tools	Cline
Bash	Cline
MCP	Cline
Browser/web tools	Cline
LLM providers	Cline
Sessions	Cline
Teams	Cline
Skills	Cline
Hooks	Cline
Checkpoints	Cline
Agent execution	Cline
Agent lifecycle control	Synapse
Authentication	Synapse
Tenants	Synapse
Policies	Synapse
Approvals	Synapse
Safety	Synapse
Verification	Synapse

Capability	Owner
Audit	Synapse
Cross-agent monitoring	Synapse
Security	Synapse
Kill switch	Synapse
Agent registry	Synapse
External-agent adapters	Synapse
World model	Synapse
Digital twins	Synapse
Simulation	Synapse
Governance	Synapse

4. apps/backend/

This is the main Synapse API server.

apps/backend/

```
|
|└─ src/
|  |
|  └─ main.ts
|  └─ app.ts
|  └─ config.ts
|  |
|  └─ server/
|     └─ http.ts
|     └─ websocket.ts
|     └─ shutdown.ts
```

- | | └─ health.ts
- | |
- | └─ routes/
- | | └─ auth.routes.ts
- | | └─ agents.routes.ts
- | | └─ sessions.routes.ts
- | | └─ tasks.routes.ts
- | | └─ teams.routes.ts
- | | └─ approvals.routes.ts
- | | └─ policies.routes.ts
- | | └─ verification.routes.ts
- | | └─ security.routes.ts
- | | └─ audit.routes.ts
- | | └─ schedules.routes.ts
- | | └─ connectors.routes.ts
- | | └─ runtimes.routes.ts
- | | └─ world.routes.ts
- | | └─ simulations.routes.ts
- | | └─ health.routes.ts
- | |
- | └─ middleware/
- | | └─ auth.ts
- | | └─ tenant.ts
- | | └─ permissions.ts
- | | └─ rate-limit.ts
- | | └─ request-id.ts
- | | └─ validation.ts


```
| | | └─ error-handler.ts
| | └─ audit.ts
| |
| └─ bootstrap/
|   └─ database.ts
|   └─ engine.ts
|   └─ events.ts
|   └─ policies.ts
|   └─ workers.ts
|
└─ tests/
```

5. main.ts

Only process startup.

main.ts

↓

loadConfig()

↓

connectDatabase()

↓

initializeSecurity()

↓

initializeClineEngine()

↓

initializeControlPlane()

↓

initializeEventBus()

↓

initializeWorkers()



startHTTPServer()



startWebSocketServer()

It should **not** contain business logic.

6. config.ts

Central configuration.

DATABASE_URL

REDIS_URL

JWT_SECRET

ENCRYPTION_KEY

SYNAPSE_ENV

CLINE_DATA_DIR

CLINE_WORKSPACE_ROOT

MAX_CONCURRENT_AGENTS

MAX_AGENT_RUNTIME

APPROVAL_TIMEOUT

VERIFICATION_TIMEOUT

WORLD_ENGINE_ENABLED

SIMULATION_ENGINE_ENABLED

LOG_LEVEL

No hardcoded secrets.

7. packages/engine-adapter/

This is one of the most important packages.

packages/engine-adapter/

```
|
| └─ src/
|   |
|   | └─ index.ts
|   |
|   | |
|   | | └─ ClineEngine.ts
|   | | └─ ClineSession.ts
|   | | └─ ClineTeam.ts
|   | | └─ ClineWorkspace.ts
|   | |
|   | | └─ lifecycle/
|   | |   |
|   | |   | └─ start.ts
|   | |   | └─ pause.ts
|   | |   | └─ resume.ts
|   | |   | └─ abort.ts
|   | |   | └─ stop.ts
|   | |   | └─ restart.ts
|   | |
|   | | └─ events/
|   | |   |
|   | |   | └─ ClineEventAdapter.ts
|   | |   | └─ event-normalizer.ts
|   | |   | └─ event-mapper.ts
```

```
| |
| |─ approvals/
| |  └─ ClineApprovalBridge.ts
| |
| |─ teams/
| |  └─ TeamManager.ts
| |    └─ TeamStateReader.ts
| |
| |─ errors/
| |    └─ ClineEngineError.ts
|
└─ tests/
```

ClineEngine.ts

This becomes Synapse's **only doorway into Cline**.

Conceptually:

```
class ClineEngine {
  createRuntime()
  startAgent()
  sendMessage()
  subscribe()
  getSession()
  getMessages()
  getUsage()
  abort()
  stop()
  delete()
  dispose()
```

```
    startTeam()
    getTeamState()
}
```

No route should directly call random Cline internals.

Bad:

route → ClineCore

Good:

route

↓

control service

↓

ClineEngine

↓

ClineCore

This prevents the backend from becoming impossible to maintain.

8. packages/control-plane/

The central orchestrator.

control-plane/

|

|— src/

| |— ControlPlane.ts

| |

| |— AgentController.ts

| |— SessionController.ts

| |— TaskController.ts

| |— TeamController.ts

| |— WorkspaceController.ts

```
| |
| | └─ commands/
| |   └─ StartAgent.ts
| |   └─ StopAgent.ts
| |   └─ AbortAgent.ts
| |   └─ PauseAgent.ts
| |   └─ ResumeAgent.ts
| |   └─ RetryAgent.ts
| |   └─ KillAgent.ts
| |
| | └─ state/
| |   └─ AgentState.ts
| |   └─ SessionState.ts
| |   └─ StateReducer.ts
| |
| | └─ errors/
| |   └─ ControlPlaneError.ts
```

The control plane answers:

What is running?

Who owns it?

What is it allowed to do?

Can it continue?

Has it completed?

Should it be stopped?

9. packages/agent-registry/

Synapse needs its own registry.

agent-registry/

|

└─ src/

| └─ AgentRegistry.ts

| └─ AgentRegistration.ts

| └─ AgentDiscovery.ts

| └─ AgentCapabilities.ts

| └─ AgentHealth.ts

| └─ AgentOwnership.ts

Database:

agents

agent_instances

agent_capabilities

agent_credentials

agent_workspaces

This is **not replacing Cline sessions**.

It identifies:

Synapse Agent

|

└─ Cline Session

10. Runtime manager

runtime-manager/

|

└─ src/

| └─ RuntimeManager.ts

| └─ RuntimeInstance.ts

```
| |— RuntimeAllocator.ts
| |— RuntimeHealthMonitor.ts
| |— RuntimeRecovery.ts
| |— WorkspaceIsolation.ts
| └─ ResourceLimits.ts
```

Responsibilities:

agent

↓

workspace

↓

runtime

↓

Cline

It handles:

- process lifecycle
- resource limits
- workspace isolation
- crash detection
- recovery
- concurrency

11. Policy engine

policy-engine/

```
|
```

```
|— src/
```

```
| |— PolicyEngine.ts
```

```
| |— PolicyEvaluator.ts
```

```
| |— PolicyCompiler.ts
```


- | |— PolicyContext.ts
- | |— PolicyDecision.ts
- | |
- | |— rules/
 - | | |— filesystem.ts
 - | | |— shell.ts
 - | | |— network.ts
 - | | |— git.ts
 - | | |— secrets.ts
 - | | |— production.ts
 - | | |— destructive.ts
- | |
- | |— schemas/
 - | | |— policy.schema.ts

Flow:

Cline wants tool



Synapse policy engine



ALLOW

BLOCK

APPROVAL

This is a major differentiator.

12. Approval engine

approval-engine/

|

- └─ src/
 - | └─ ApprovalEngine.ts
 - | └─ ApprovalRequest.ts
 - | └─ ApprovalResolver.ts
 - | └─ ApprovalTimeout.ts
 - | └─ ApprovalPolicy.ts
 - | └─ ApprovalAudit.ts

Flow:

tool request



policy



requires approval



DB



WebSocket



human



approve/reject



Cline callback

Never allow an approval request to remain indefinitely pending.

13. Safety engine

safety-engine/

|

```
└─ src/
|   └─ SafetyEngine.ts
|   └─ RiskClassifier.ts
|   └─ ActionRisk.ts
|   └─ BlastRadius.ts
|   └─ SecretDetector.ts
|   └─ PromptInjectionDetector.ts
|   └─ CommandRiskAnalyzer.ts
|   └─ WorkspaceBoundary.ts
|   └─ NetworkGuard.ts
|   └─ KillSwitch.ts
```

Before consequential execution:

ACTION

↓

risk analysis

↓

blast radius

↓

policy

↓

verification requirements

↓

allow / approval / block

14. Verification engine

This should be a **first-class subsystem**, not a few tests.

verification-engine/

```

|
| └─ src/
|   |
|   | └─ VerificationEngine.ts
|   |
|   | └─ VerificationPlan.ts
|   |
|   | └─ VerificationRunner.ts
|   |
|   | └─ VerificationResult.ts
|   |
|   |
|   | └─ verifiers/
|   |   |
|   |   | └─ FileVerifier.ts
|   |   |
|   |   | └─ GitVerifier.ts
|   |   |
|   |   | └─ TestVerifier.ts
|   |   |
|   |   | └─ BuildVerifier.ts
|   |   |
|   |   | └─ APIVerifier.ts
|   |   |
|   |   | └─ DatabaseVerifier.ts
|   |   |
|   |   | └─ SecurityVerifier.ts
|   |   |
|   |   | └─ ContractVerifier.ts
|   |   |
|   |   |
|   |   | └─ evidence/
|   |   |   |
|   |   |   | └─ EvidenceStore.ts
|   |   |   |
|   |   |   | └─ EvidenceHasher.ts
|   |   |   |
|   |   |   | └─ EvidenceChain.ts
|   |   |   |
|   |   |   |
|   |   |   |
|   |   |   | └─ policies/
|   |   |   |   |
|   |   |   |   | └─ verification-policy.ts

```

The crucial architecture:

Agent says:

"I fixed it."



Synapse does NOT trust this.



Independent verification



Evidence



PASS / FAIL / INCONCLUSIVE

15. Audit engine

audit-engine/

|

|— src/

| |— AuditEngine.ts

| |— AuditWriter.ts

| |— AuditReader.ts

| |— AuditHasher.ts

| |— AuditRetention.ts

| |— AuditExporter.ts

Every important operation gets:

who

what

when

where

why

policy

decision

result

evidence

16. Event bus

event-bus/

|

|— src/

| |— EventBus.ts

| |— EventPublisher.ts

| |— EventSubscriber.ts

| |— EventStore.ts

| |— EventTypes.ts

| |— EventReplay.ts

Cline events:

content_start

content_update

content_end

iteration_start

iteration_end

usage

notice

done

error

become Synapse events:

cline.*

agent.*

policy.*

approval.*

verification.*

security.*

world.*

simulation.*

17. Observability

observability/

|

|— src/

| |— Metrics.ts

| |— Tracing.ts

| |— Logging.ts

| |— AgentTelemetry.ts

| |— RuntimeTelemetry.ts

| |— CostTelemetry.ts

| |— HealthSignals.ts

Track:

tokens

cost

runtime

iterations

tool calls
errors
approvals
rejections
verification failures
retries
CPU
memory
network

18. Scheduler

Don't blindly copy Cline's scheduler.

Synapse needs a governance-aware scheduler.

scheduler/

```
|  
| └─ src/  
|   └─ Scheduler.ts  
|   └─ ScheduleRepository.ts  
|   └─ ScheduleRunner.ts  
|   └─ ScheduleLock.ts  
|   └─ SchedulePolicy.ts  
|   └─ ScheduleRecovery.ts  
|   └─ ScheduleExecution.ts
```

Architecture:

Schedule

↓

Policy

↓

Create Cline execution



Monitor



Verify



Audit

Cline already has schedule/hub infrastructure, so Synapse should reuse it where appropriate rather than creating a second execution engine. Cline's package documentation explicitly places scheduled execution and schedule persistence in its core hub services.

19. Connector manager

Cline's CLI currently has connector functionality and integrations; preserve the underlying capability but put Synapse governance around it.

connector-manager/

|

|— src/

| |— ConnectorManager.ts

| |— ConnectorRegistry.ts

| |— ConnectorAdapter.ts

| |— ConnectorAuth.ts

| |— ConnectorPolicy.ts

| |— ConnectorHealth.ts

| |— adapters/

| |— telegram/

| |— slack/

| |— discord/

| |— whatsapp/

```
|   ├── gchat/
|   └── linear/
```

20. External agent control

This is the part that addresses your requirement that Synapse eventually control agents from **other companies**, while Cline remains Synapse's own execution engine.

Do **not** pretend external agents are Cline sessions.

Create:

agent-adapters/

```
|
|   ├── src/
|   |   ├── AgentAdapter.ts
|   |   ├── AgentAdapterRegistry.ts
|   |   ├── ExternalAgent.ts
|   |   ├── ExternalSession.ts
|   |   |
|   |   ├── protocols/
|   |   |   ├── ACPAdapter.ts
|   |   |   ├── MCPAdapter.ts
|   |   |   ├── HTTPAgentAdapter.ts
|   |   |   └── WebSocketAgentAdapter.ts
|   |   |
|   |   └── capabilities/
|   |       ├── observe.ts
|   |       ├── pause.ts
|   |       ├── resume.ts
|   |       ├── stop.ts
|   |       └── approve.ts
```

| └─ verify.ts

But:

Cline = fully controlled

External agents = capability-dependent controlled

If an external agent cannot be stopped, Synapse must explicitly show:

CONTROL CAPABILITY:

STOP ✗

PAUSE ✗

OBSERVE ✓

VERIFY ✓

Never fake control.

21. World Engine

This is your digital-twin system.

world-engine/

|

|─ src/

| |─ WorldEngine.ts

| |

| |─ ingestion/

| | |─ DataIngestion.ts

| | |─ CSVImporter.ts

| | |─ JSONImporter.ts

| | |─ DatabaseImporter.ts

| | |─ APIImporter.ts

| | |─ LogImporter.ts

| | └─ EventStreamImporter.ts

- | |
- | |— model/
 - | | |— WorldModel.ts
 - | | |— Entity.ts
 - | | |— Relationship.ts
 - | | |— State.ts
 - | | |— Event.ts
 - | | |— Constraint.ts
- | |
- | |— extraction/
 - | | |— EntityExtractor.ts
 - | | |— RelationshipExtractor.ts
 - | | |— DependencyExtractor.ts
 - | | |— SchemaInference.ts
- | |
- | |— twin/
 - | | |— DigitalTwin.ts
 - | | |— TwinBuilder.ts
 - | | |— TwinSynchronizer.ts
 - | | |— TwinSnapshot.ts
- | |
- | |— queries/
 - | | |— WorldQuery.ts
 - | | |— DependencyQuery.ts

22. Simulation engine

Separate from visualization.

simulation-engine/

|

|— src/

| |— SimulationEngine.ts

| |— SimulationRun.ts

| |— Scenario.ts

| |— ScenarioBuilder.ts

| |— SimulationClock.ts

| |— EventSimulator.ts

| |— StateTransition.ts

| |— RuleEngine.ts

| |— ConstraintEngine.ts

| |— AgentSandbox.ts

| |— MonteCarloRunner.ts

| |— ResultAnalyzer.ts

| |— ComparisonEngine.ts

Flow:

real world



snapshot



clone



modify variables



simulate



observe



compare with baseline

23. Twin engine

twin-engine/

|

|— src/

| |— TwinEngine.ts

| |— TwinRegistry.ts

| |— TwinLifecycle.ts

| |— TwinVersioning.ts

| |— TwinSync.ts

| |— TwinDiff.ts

| |— TwinConfidence.ts

Important:

Every twin should have:

source

timestamp

version

confidence

freshness

assumptions

coverage

So Synapse never presents an inferred model as ground truth.

24. Database layer

packages/database/

```
|
|
| └─ src/
|
|   └─ client.ts
|
|   └─ transaction.ts
|
|   └─ repositories/
|
|     └─ AgentRepository.ts
|
|     └─ SessionRepository.ts
|
|     └─ TaskRepository.ts
|
|     └─ PolicyRepository.ts
|
|     └─ ApprovalRepository.ts
|
|     └─ AuditRepository.ts
|
|     └─ VerificationRepository.ts
|
|     └─ ScheduleRepository.ts
|
|     └─ ConnectorRepository.ts
|
|     └─ WorldRepository.ts
|
|     └─ SimulationRepository.ts
|
|
| └─ schemas/
|
|   └─ agents.ts
|
|   └─ sessions.ts
|
|   └─ tasks.ts
|
|   └─ policies.ts
|
|   └─ approvals.ts
|
|   └─ audits.ts
|
|   └─ verification.ts
|
|   └─ schedules.ts
|
|   └─ worlds.ts
```

25. Database source-of-truth rule

Very important.

Cline owns:

agent execution state

messages

Cline session data

Cline team state

Cline runtime state

Synapse owns:

tenant

agent identity

authorization

policies

approvals

governance

audit

verification

external mappings

world models

simulation metadata

Do **not** duplicate Cline's entire session database into PostgreSQL.

Cline's current SDK explicitly provides stateful session/storage functionality.

26. Contracts package

contracts/

|


```
└─ src/
|   └─ agent.ts
|   └─ session.ts
|   └─ events.ts
|   └─ commands.ts
|   └─ policy.ts
|   └─ approval.ts
|   └─ verification.ts
|   └─ security.ts
|   └─ world.ts
|   └─ simulation.ts
```

Use schemas such as Zod.

Every API boundary gets validation.

27. Security package

```
security/
|
└─ src/
|   └─ authentication/
|   └─ authorization/
|   └─ encryption/
|   └─ secrets/
|   └─ workspace/
|   └─ network/
|   └─ sandbox/
|   └─ rate-limit/
|   └─ threat-detection/
```

| └─ security-events/

Never let frontend directly access:

API keys

database credentials

Cline internal storage

workspace filesystem

runtime processes

28. Secrets

secrets/

|

└─ src/

| └─ SecretManager.ts

| └─ Encryption.ts

| └─ SecretResolver.ts

| └─ SecretRedactor.ts

| └─ SecretRotation.ts

API:

store()

resolve()

rotate()

revoke()

redact()

Logs must pass through the redactor.

29. Tenancy

tenancy/

|

```
└─ src/
|   └─ TenantContext.ts
|   └─ TenantResolver.ts
|   └─ TenantIsolation.ts
|   └─ TenantLimits.ts
|   └─ TenantRepository.ts
```

Every request:

request

↓

identity

↓

tenant

↓

authorization

↓

resource

No cross-tenant resource access.

30. Worker architecture

apps/worker/

|

└─ src/

| └─ main.ts

| |

| └─ workers/

| | └─ agent-worker.ts

| | └─ verification-worker.ts

- | | |— simulation-worker.ts
- | | |— audit-worker.ts
- | | |— telemetry-worker.ts
- | |
- | |— queues/
- | | |— agent.queue.ts
- | | |— verification.queue.ts
- | | |— simulation.queue.ts
- | |
- | |— recovery/
- | | |— retry.ts
- | | |— dead-letter.ts
- | | |— recovery.ts

The HTTP server should not become the place where long-running simulations and agent operations live.

31. API architecture

POST /api/agents

GET /api/agents

GET /api/agents/:id

POST /api/agents/:id/start

POST /api/agents/:id/pause

POST /api/agents/:id/resume

POST /api/agents/:id/abort

POST /api/agents/:id/stop

POST /api/agents/:id/kill

GET /api/sessions

GET /api/sessions/:id

GET /api/sessions/:id/messages

GET /api/sessions/:id/usage

GET /api/approvals

POST /api/approvals/:id/approve

POST /api/approvals/:id/reject

GET /api/policies

POST /api/policies

PUT /api/policies/:id

POST /api/verifications

GET /api/verifications/:id

GET /api/audit

POST /api/schedules

GET /api/schedules

GET /api/worlds

POST /api/worlds

GET /api/worlds/:id

POST /api/worlds/:id/simulations

GET /api/simulations/:id

GET /api/simulations/:id/results

32. WebSocket architecture

One connection per application.

/ws



Never:

ApprovalModal → WebSocket

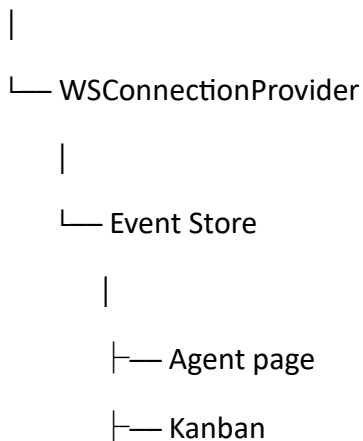
AgentPage → WebSocket

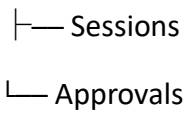
SessionPage → WebSocket

Kanban → WebSocket

Instead:

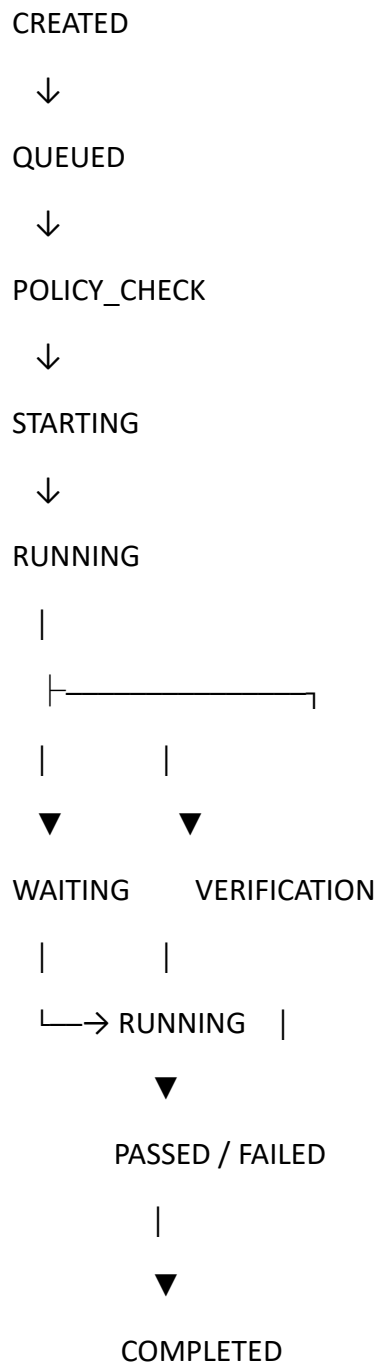
App





33. Agent lifecycle

This should be the canonical lifecycle.



Failure:

RUNNING

↓

ERROR

↓

RECOVERY

↓

RETRY

Kill:

ANY ACTIVE STATE

↓

KILL REQUEST

↓

SAFETY CHECK

↓

CLINE ABORT/STOP

↓

PROCESS CONFIRMATION

↓

KILLED

34. Verification lifecycle

Agent completion

↓

Freeze relevant state

↓

Collect evidence

↓

Run verification plan

↓

Tests



Build



Git diff



Contracts



Security



Behavioral checks



Verdict

Possible result:

PASS

FAIL

INCONCLUSIVE

Never reduce everything to boolean success.

35. World/agent integration

This is where your more ambitious idea comes together.

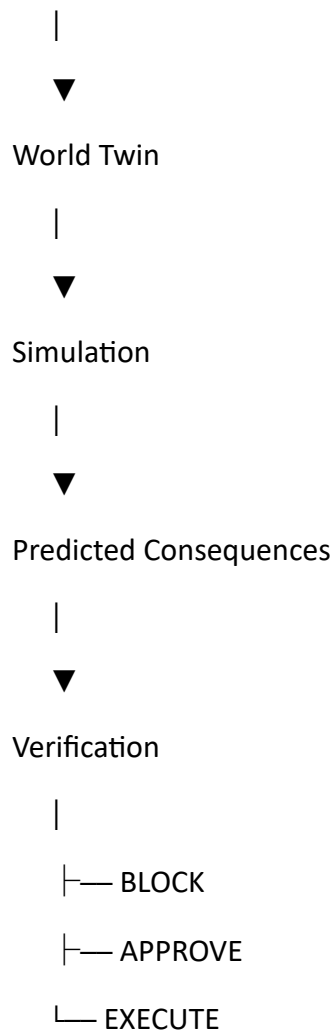
Cline Agent



Proposed Action



Synapse Safety

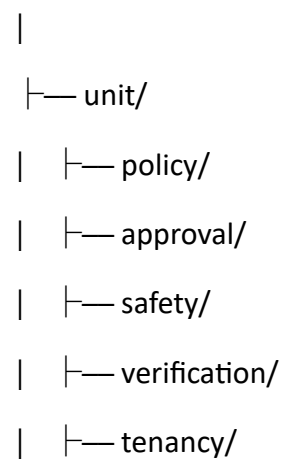


That gives Synapse something much more interesting than a Cline dashboard.

36. Tests

Make tests a first-class architecture layer.

tests/



- | └─ world/
- |
- | └─ integration/
- | └─ cline-engine/
- | └─ database/
- | └─ websocket/
- | └─ policy/
- | └─ verification/
- |
- | └─ e2e/
- | └─ agent-lifecycle/
- | └─ approval/
- | └─ kill-switch/
- | └─ verification/
- | └─ simulation/
- |
- | └─ security/
- | └─ tenant-isolation/
- | └─ secret-leak/
- | └─ command-injection/
- | └─ path-traversal/
- | └─ prompt-injection/
- |
- └─ chaos/
 - └─ runtime-crash/
 - └─ database-loss/
 - └─ websocket-loss/

└─ worker-crash/

└─ agent-timeout/

37. Most important implementation rule for the coding agent

Give Antigravity/AGY this rule:

Never invent a Cline API.

Before using a Cline function:

search upstream source

↓

find implementation

↓

find type definition

↓

find tests

↓

use actual API

Cline's own repository notes that SDK packages resolve through compiled dist/ and need SDK rebuilding after source/dependency changes, so the development workflow must account for that rather than letting an agent assume hot reload works.

38. Another critical rule: don't duplicate Cline

AGY should **not** create:

SynapseAgentRuntime

SynapseToolRuntime

SynapseSessionEngine

SynapseLLMEngine

SynapseTeamEngine

that replicate Cline.

Instead:

Synapse



ClineEngineAdapter



Cline

Synapse adds **control**, not another agent runtime.

39. Cline CLI

Because we're forking the repository, the CLI is no longer something the customer installs.

It becomes an **internal capability**.

Cline's CLI currently exposes commands including mcp, dev, doctor, history, hook, plugin, schedule, hub, update, version, and kanban.

We should therefore preserve the useful Cline capabilities in the fork, but Synapse's backend decides **when/how they are exposed**.

For example:

Cline CLI



Synapse Runtime



Synapse Policy



Cline execution

not:

User → arbitrary shell → Cline

40. Cline Kanban

Same principle.

Don't make a completely separate Synapse execution system just because Cline has a Kanban application.

Cline's CLI itself currently exposes kanban.

Synapse should instead treat Kanban as:

Synapse Task



Cline Session / Team



Cline task execution



Synapse events



Synapse verification

Later the **Synapse frontend** can provide the superior Kanban UI.

41. Cline Teams

Use Cline Teams rather than inventing Synapse's own multi-agent runtime.

Synapse Team



Cline Team



Cline teammates



Cline task board / mailbox / mission state



Synapse observes



Synapse governs



Synapse verifies

This is one of the places where forking Cline gives you a major advantage.

42. Plugins and hooks

Don't throw these away.

Cline's current SDK/CLI architecture explicitly supports plugins and hooks.

Synapse should have:

packages/control-extensions/

|

|— src/

| |— ClinePluginBridge.ts

| |— ClineHookBridge.ts

| |— ToolPolicyHook.ts

| |— AuditHook.ts

| |— SecurityHook.ts

| |— VerificationHook.ts

The important distinction:

Cline hook



Synapse interception



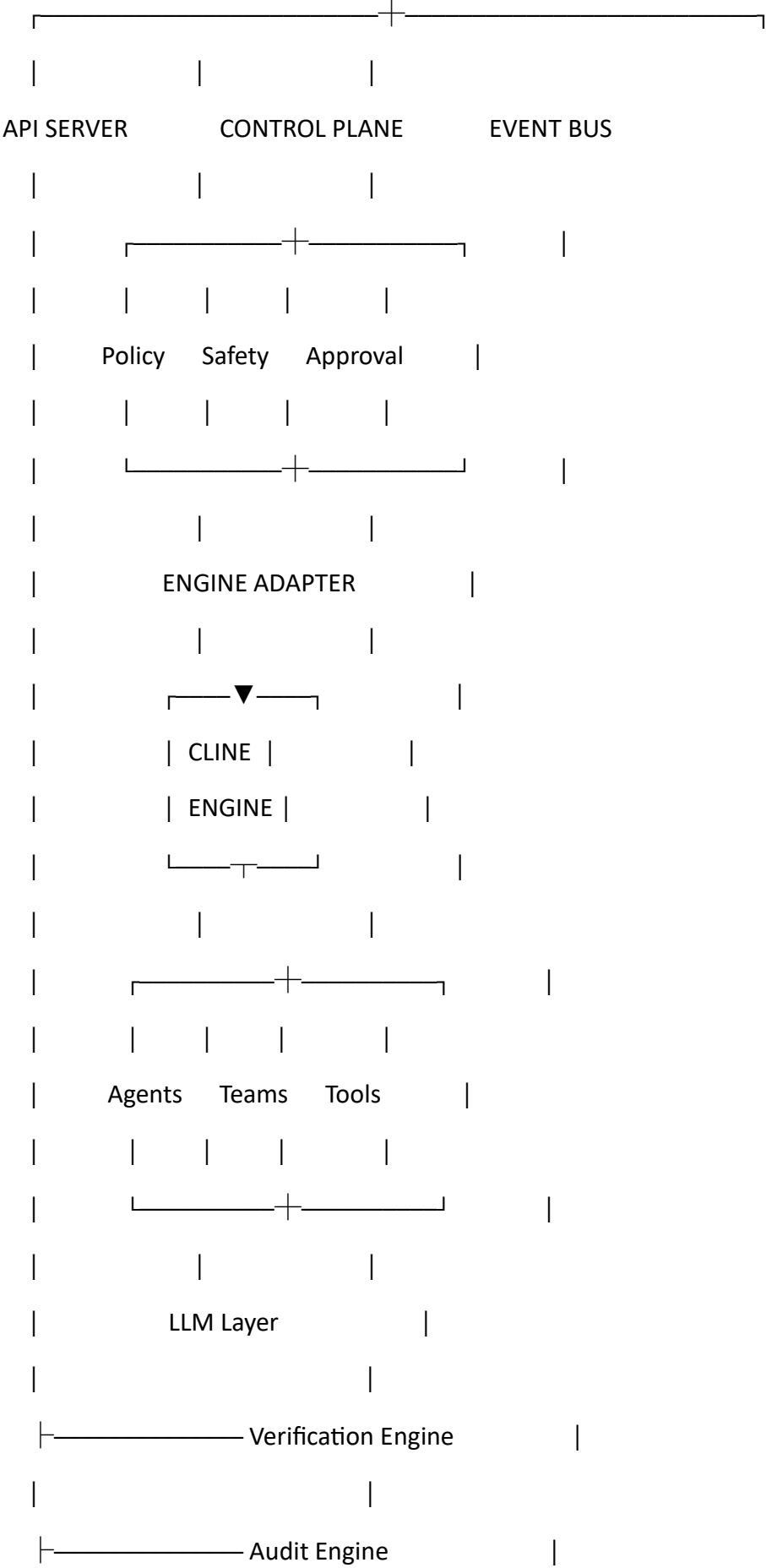
policy / audit / safety

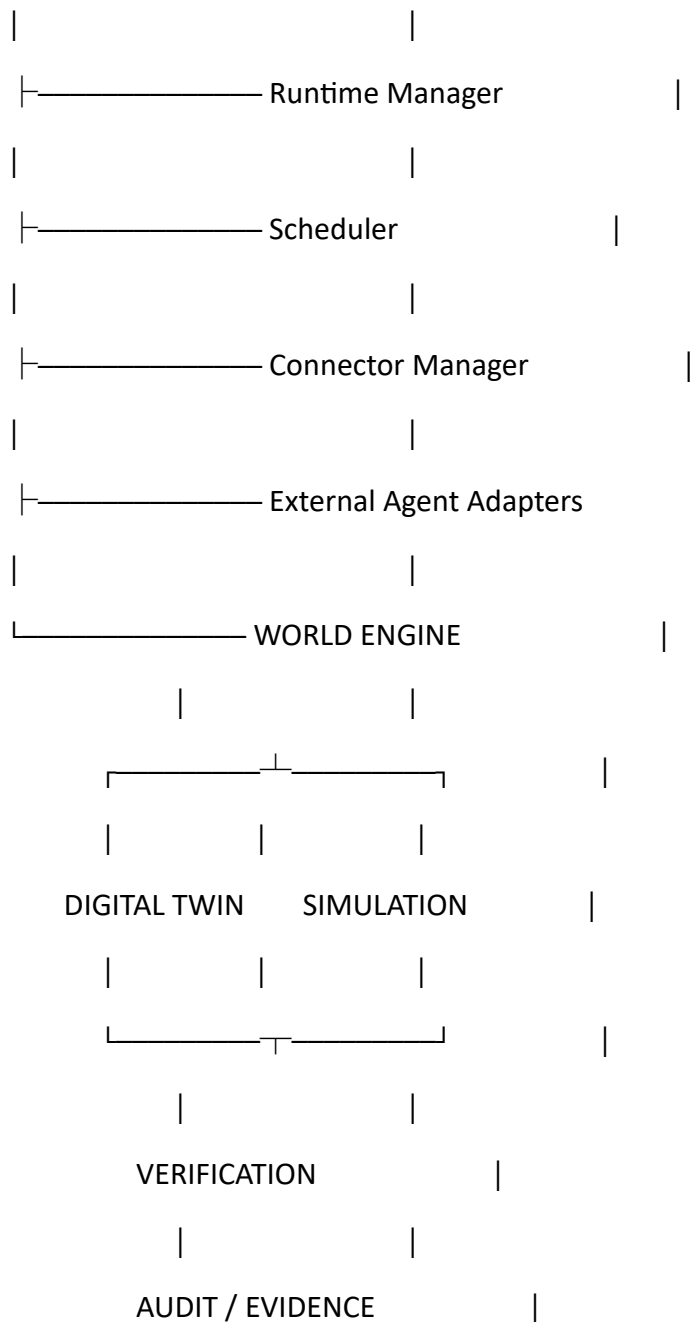
43. Final backend architecture

The whole system becomes:

SYNAPSE BACKEND

|





The key idea

Cline is the brain/execution engine.

Synapse is the operating/control system around that brain.

And the third layer is:

World Engine = model of the environment in which the agent operates.

So the eventual loop is:

PERCEIVE



CLINE REASON



PROPOSE ACTION



SYNAPSE POLICY



SAFETY



SIMULATE / DIGITAL TWIN



APPROVAL IF REQUIRED



CLINE EXECUTES



OBSERVE



VERIFY



AUDIT



UPDATE WORLD MODEL



NEXT ACTION